

Step 2 — Database Persistence — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — The table, and the request ID](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The table, and the request ID

Basic Version

A database table here is just a structured way to remember what a `POST /run-task` request did, so it survives after the request finishes and the server eventually restarts. One row per run: what topic was asked for, what came back, whether it succeeded, and when it happened.

SQLite is the right tool for this step — it's a single file on disk, needs no separate server process, and Python's standard library (or `sqlalchemy`) talks to it directly. Don't reach for anything heavier yet.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Design the `Run` table's columns to match what the README's checklist actually asks for: `id` (a UUID, not an auto-increment integer — you don't want run IDs to be guessable/sequential), `input` (the topic), `output` (the final report), `status` (`"pending"` , `"success"` , `"failed"`), `log` (a text field for anything worth keeping — could start small), `created_at` , `completed_at` .

`GET /runs/{id}` is a second, simple route: look up that ID, return `404` if it doesn't exist, return the row's data if it does. The ID for each request should be generated once, at the very top of the route (before calling the graph), and threaded through every log line for that request — this is what makes "find every log line for this one run" actually possible later, instead of a wall of interleaved logs from every concurrent request with no way to tell them apart.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The README's own Step 2 test is the whole point here: "kill the database connection in the middle of a request... confirm a clean 500, not a raw error trace." Read that literally — it means your database write has to be wrapped the same way Step 1's Advanced hint wrapped `graph.invoke(...)`: assume it can fail, and decide on purpose what the client sees when it does.

There's a subtler version of the same problem, specific to this step: what happens to a run's *status* if the graph itself succeeds, but the database write that would mark it "success" fails right after? Right now, nothing — that run would be stuck reading "pending" forever, even though the actual work finished fine. The real design is: write a "pending" row *before* calling the graph (so a lookup mid-run returns something sensible), then update it to "success" or "failed" afterward, and if *that* update itself fails, that's a database problem worth its own alert, separate from whether the graph run succeeded.

The extra pieces:

- Wrap every database read/write in its own `try/except`, distinct from the `try/except` around `graph.invoke(...)` — a graph failure and a database failure are different problems and deserve different log messages, even if the client sees the same generic 500 either way.
- Write the `Run` row as "pending" *before* invoking the graph, then update it after — so `GET /runs/{id}` called mid-run returns a real, honest "pending" status instead of a 404 for a run that's genuinely in progress.
- Use a context manager (`with Session()` as `session:` if using SQLAlchemy, or `with sqlite3.connect(...)` as `conn:`) for every database operation, so a connection is never left open/leaked if an exception happens partway through — this is what actually prevents the "database connections leak under repeated requests" problem in this project's own problems table.

Sketch the two-phase write ("pending" then update) yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate assume every database operation succeeds and that a run's status only needs setting once, at the end. Advanced treats the database itself as something that can fail independently of the graph, and makes a run's status honest at every point in its lifecycle — not just correct if nothing goes wrong.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
db/models.py:
    Run table: id, input, output, log, status, created_at, completed_at

db/database.py:
    set up the SQLite connection/engine

routes.py:
    POST /run-task(request):
        run_id = new UUID
        save a Run row with status "pending"
        result = run the graph
        update the Run row: output, status "success", completed_at
        return the result

    GET /runs/{run_id}:
        look up the Run row by id
        if not found: return 404
        return its data

test: kill the database mid-request (see Doc12's Break-It exercise),
confirm you get a clean 500, not a raw traceback
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
db/models.py:
class Run(Base):
    id: str (UUID, primary key)
    input: str
    output: str | None
    log: str | None
    status: str # "pending" | "success" | "failed"
    created_at: datetime
    completed_at: datetime | None

db/database.py:
engine = create_engine("sqlite:///runs.db")
SessionLocal = sessionmaker(bind=engine)
Base.metadata.create_all(engine)

api/routes.py:
@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest):
    run_id = str(uuid4())
    logger.info("run %s starting for topic=%s", run_id, request.topic)
    with SessionLocal() as session:
        session.add(Run(id=run_id, input=request.topic, status="pending", created_at=now))
        session.commit()

    result = graph.invoke(make_initial_state(request.topic))

    with SessionLocal() as session:
        run = session.get(Run, run_id)
        run.output = result["final_report"]
        run.status = "success"
        run.completed_at = utcnow()
        session.commit()

    return TaskResponse(final_report=result["final_report"], approved=result["approved"])

@router.get("/runs/{run_id}")
def get_run(run_id: str):
    with SessionLocal() as session:
        run = session.get(Run, run_id)
    if not run:
```

```
        raise HTTPException(status_code=404, detail="Run not found")
    return run
```

Test `GET /runs/{id}` against a real run's ID, and confirm every log line for that request includes `run_id`. Write the rest yourself, then compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's almost the hardened version — fill in the missing except block yourself:

```
@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest) -> TaskResponse:
    run_id = str(uuid4())
    logger.info("run %s starting for topic=%s", run_id, request.topic)

    try:
        with SessionLocal() as session:
            session.add(Run(id=run_id, input=request.topic, status="pending", created_at=datetime.utcnow()))
            session.commit()
    except Exception:
        logger.exception("run %s: failed to write initial pending row", run_id)
        raise HTTPException(status_code=500, detail="Could not start task – please try again")

    try:
        result = graph.invoke(make_initial_state(request.topic))
    except Exception:
        logger.exception("run %s: graph execution failed", run_id)
        # your turn: try to update the Run row's status to "failed" here too
        # (wrapped in its own try/except – this write can ALSO fail),
        # then raise HTTPException(status_code=500, ...)
        ...

    # your turn: wrap the final "success" update in its own try/except,
    # the same shape as the "pending" write above
    ...

    return TaskResponse(final_report=result["final_report"], approved=result["approved"])
```

Fill in both missing pieces, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same `Run` table and two routes at 3 completeness levels — Basic and Intermediate write to the database as if it always succeeds and only need to set a run's status once. Advanced treats every database call as something that can independently fail, at 3 separate points (the initial write, the graph call, the final update), and writes a "pending" row up front so `GET /runs/{id}` is honest about a run that's still in progress instead of returning `404` for something that's actually happening right now.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)